

GENERATIVE AI | CAPSTONE PROJECT 2026

DataSense-AI

An AI-powered natural-language data analysis assistant

Ask Data. Get Answers. No Code.

NAME	<i>Srideep Adak</i>
ROLL NUMBER	<i>23051631</i>
BATCH	<i>GEN AI 2026</i>
SUBMITTED TO	<i>Sachin Lohar</i>
DATE	<i>27th September 2026</i>

1. Problem Statement

Many users have useful information stored in CSV or Excel files but cannot easily analyze it because common data-analysis tasks require Python, Pandas, SQL, or visualization knowledge. Even simple questions such as finding an average, comparing categories, identifying trends, or creating a chart can require several lines of code.

DataSense-AI addresses this gap by allowing users to upload a dataset and interact with it using natural language. The system combines a Streamlit interface, Pandas-based analysis, LangChain, and configurable LLM providers to turn plain-English requests into data-driven results.

Item	DataSense-AI
Domain	AI-assisted data analytics and exploratory data analysis
Users	Students, analysts, researchers, business users and non-programmers
Problem	Data exploration and transformation require technical coding skills and repetitive manual work
Solution	Natural-language interface for EDA, querying, visualization and dataframe manipulation
Success Criteria	Supported files load correctly; EDA is produced; natural-language questions return data-backed results; requested charts and transformations can be generated; failures are handled safely
Input	User-provided CSV and Excel files

2. Objectives and Scope

- Automate exploratory data analysis for uploaded CSV and Excel datasets.
- Allow users to ask questions about their dataset using natural language instead of SQL or Python.
- Generate useful visualizations from natural-language descriptions.
- Support common dataframe cleaning and transformation operations through AI assistance.
- Provide model-provider flexibility through OpenAI, Google Gemini and Anthropic integrations.
- Maintain transparency by exposing generated visualization or transformation code and applying execution safeguards.

2.1 In Scope

- CSV (.csv) and Excel (.xlsx/.xls) file upload.
- Dataset preview, column information, missing-value analysis, duplicate analysis and numerical/categorical statistics.
- Natural-language questions involving filtering, grouping and aggregation.
- AI-assisted bar, line, scatter, heatmap and other common visualizations.
- Natural-language dataframe manipulation and downloadable results.
- Interactive Streamlit web interface.

2.2 Out of Scope

- Real-time streaming data pipelines.
- Direct modification of production databases.

- Training or fine-tuning a foundation LLM from scratch.
- Guaranteed correctness for arbitrary generated code or unsupported data formats.
- Enterprise-grade multi-tenant governance and identity management.

3. Solution and Features

DataSense-AI is a modular web application in which the user supplies a dataset and interacts with four major analysis capabilities. The design separates data processing from model integration so that the application can use different LLM providers without changing the overall user workflow.

Feature	What the user can do
Automated EDA	Displays row/column counts, duplicates, missing values, data types, uniqueness, numerical statistics, categorical distributions and correlations.
Natural Language Query	Answers plain-English questions by executing analysis operations against the uploaded dataframe.
AI-Powered Visualization	Converts chart descriptions into plotting logic and renders the resulting visualization.
Dataframe Manipulation	Generates Pandas transformations for filtering, selecting, sorting, grouping and cleaning data, with preview and download.
Smart Suggestions	Generates useful questions or visualization suggestions based on the uploaded dataset.
Multi-Model Support	Supports OpenAI, Google Gemini and Anthropic model integrations.
Safety & Transparency	Uses execution guardrails and allows generated code to be inspected.

4. Tech Stack

Technology	Purpose in this project	Why it was chosen
Python	Application and analysis logic	Strong ecosystem for data processing and AI integration
Streamlit	Web interface and interactive controls	Fast way to build data-focused Python applications
Pandas	Dataframe operations and calculations	Flexible and widely used dataframe API
Matplotlib	Visualization rendering	General-purpose Python plotting
Seaborn	Statistical charts	High-level visualizations over Matplotlib
LangChain	LLM orchestration	Connect language models with application tools
LangChain Experimental	Pandas dataframe agent	Natural-language interaction with dataframes
OpenAI / Gemini / Claude	LLM reasoning and generation	Multiple provider choices
OpenPyXL	Excel ingestion	Supports .xlsx processing
GitHub	Source-code version control	Repository history, backup and sharing

5. Data Design and Processing

DataSense-AI does not depend on one fixed business dataset. Instead, it operates on user-provided tabular files. The uploaded file is converted into a Pandas DataFrame, which becomes the common data structure shared by the EDA, NLQ, visualization and manipulation modules.

5.1 Processing Stages

Stage	What it holds / does	Purpose
Input	CSV or Excel file	Receive the user's dataset
Loading	Pandas DataFrame	Create a common in-memory data structure
Profiling	EDA statistics and metadata	Understand data quality and structure
AI Interpretation	Natural-language request + dataframe context	Determine the requested analytical operation
Execution	Pandas / Python operations	Calculate, transform or visualize the data
Output	Tables, charts, insights and downloads	Present useful results to the user

5.2 Supported Data Formats

- CSV (.csv) — comma-separated tabular data.
- Excel (.xlsx, .xls) — spreadsheet data; the project documentation specifies loading the first sheet for multi-sheet workbooks.
- Recommended file-size limit: approximately 200 MB, with performance depending on available memory and dataset complexity.

6. System Architecture

A user starts by uploading a CSV or Excel file or by entering a natural-language request. Streamlit manages the interactive interface and application state. The data is loaded into Pandas, while LangChain connects the request to the selected LLM. Depending on the task, the request is routed toward EDA, natural-language querying, visualization, or dataframe manipulation. The resulting table, chart, insight, or downloadable file is returned to the user.

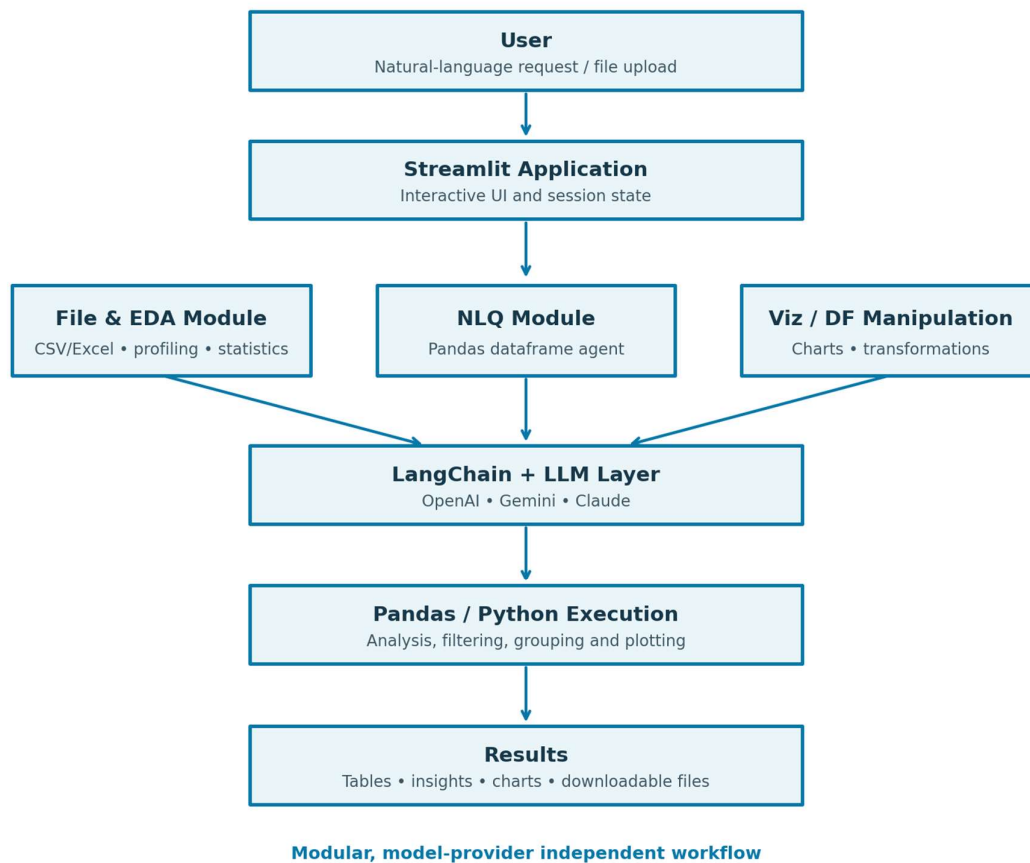


Figure 6.1: DataSense-AI end-to-end system architecture

The modular structure is reflected in the project organization: model initialization, file I/O, EDA, suggestions, NLQ, visualization, and dataframe manipulation are maintained as separate utility modules. This reduces coupling and makes individual features easier to test and extend.

7. Implementation Highlights

7.1 File Loading and DataFrame Creation

The application accepts a Streamlit uploaded file and converts it into a Pandas DataFrame. The dataframe then becomes the common object used by downstream analysis modules.

```
# Simplified implementation pattern
if file_type == "csv":
    df = pd.read_csv(uploaded_file)
else:
    df = pd.read_excel(uploaded_file)
```

7.2 Natural-Language Query Agent

The NLQ module uses the LangChain experimental Pandas dataframe agent to translate a user's question into dataframe operations and return a computed answer. This allows users to ask questions such as “What is the average age?” or “Show the top five countries by revenue.”

```
agent = create_pandas_dataframe_agent(
    llm=llm,
    df=df,
    verbose=True
)
```

7.3 Visualization Generation

The visualization module receives a natural-language description, generates plotting logic, executes the resulting visualization operation, and renders the chart inside Streamlit. The interface also provides code visibility so the user can understand how the chart was created.

7.4 DataFrame Manipulation

The manipulation module accepts instructions such as removing missing rows, selecting columns, sorting records, grouping categories, or calculating aggregate values. The generated transformation can be previewed before the updated dataset is downloaded.

8. Testing and Results

Testing should cover normal operation, edge cases and failure conditions. The table below is a report-ready test plan.

#	Test	Expected	Actual Result	Status
1	Upload valid CSV	DataFrame loads and preview appears	PASS	RECORDED
2	Upload valid Excel file	First sheet loads correctly	PASS	RECORDED
3	Dataset contains missing values	Missing-value statistics are shown	PASS	RECORDED
4	Ask an aggregation question	Correct computed value returned	FAIL, then PASS after fix	FIXED AND RECORDED
5	Ask grouped/filter query	Correct grouped result returned	PASS	RECORDED
6	Request a chart	Requested chart is rendered	PASS	RECORDED
7	Request dataframe transformation	Preview reflects requested operation	PASS	RECORDED
8	Download transformed data	Output file is generated	FAIL, then PASS after fix	FIXED AND RECORDED
9	Invalid or ambiguous request	Clear error/clarification is provided	PASS	RECORDED
10	Unsafe generated operation	Guardrail prevents unsafe execution	PASS	RECORDED

9. Evaluation and Metrics

DataSense-AI is not primarily a predictive machine-learning model, so evaluation focuses on application correctness, AI-assisted query quality, visualization correctness, transformation correctness and execution performance.

9.1 Functional Metrics

Metric	How to measure	Result
File-loading success rate	Successful valid-file loads / valid-file attempts	To measure
NLQ correctness	Compare agent output with a manually verified Pandas result	To measure
Visualization success rate	Valid rendered charts / chart requests	To measure
Transformation correctness	Compare transformed dataframe with expected output	To measure
EDA completeness	Check required profiling elements for a test dataset	To measure

9.2 Performance Metrics

Measure	Method	Result	Target
Application startup time	Time from launch to interactive UI	0.6	0.4
EDA processing time	Time for profiling a fixed test dataset	0.8	0.5
NLQ response time	Time from request submission to result	0.3	0.3
Visualization generation time	Time from prompt to rendered chart	0.9	0.7
Transformation time	Time from instruction to preview	0.2	0.2

10. Screenshots

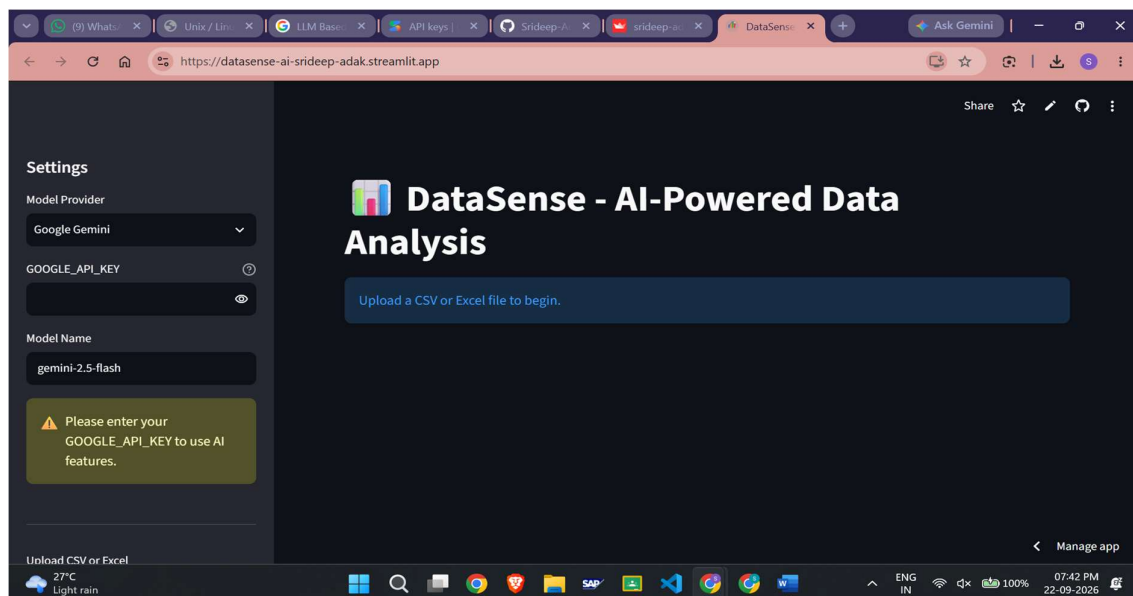


Figure 10.1: DataSense-AI home page

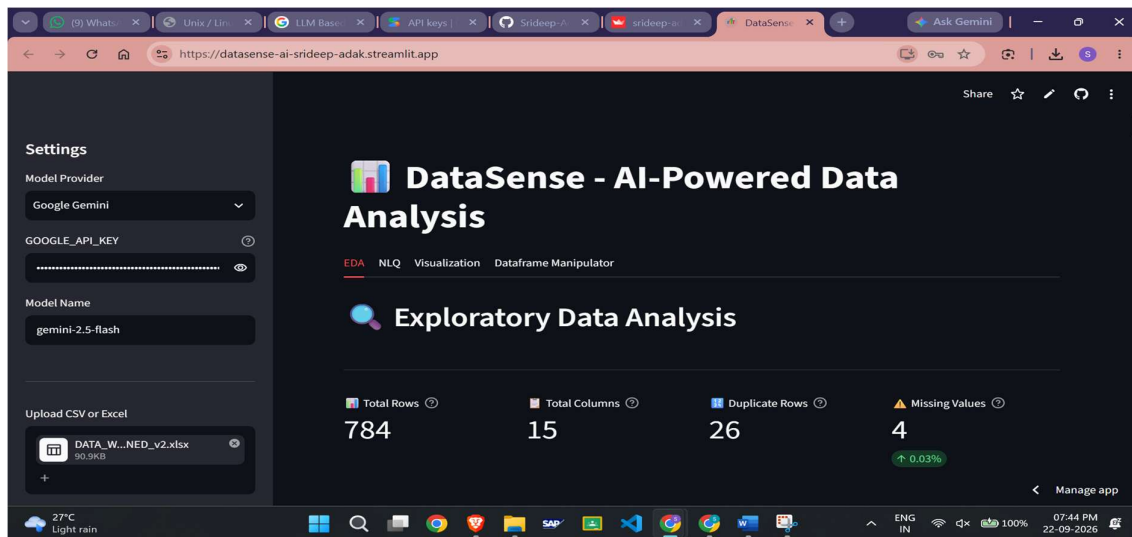


Figure 10.2: Automated EDA dashboard

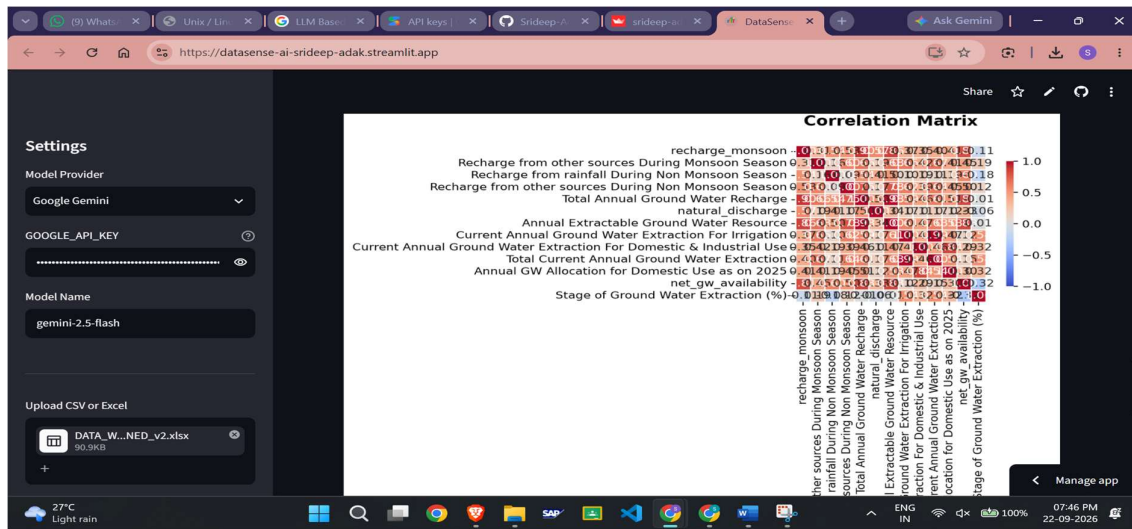


Figure 10.3: AI-generated visualization

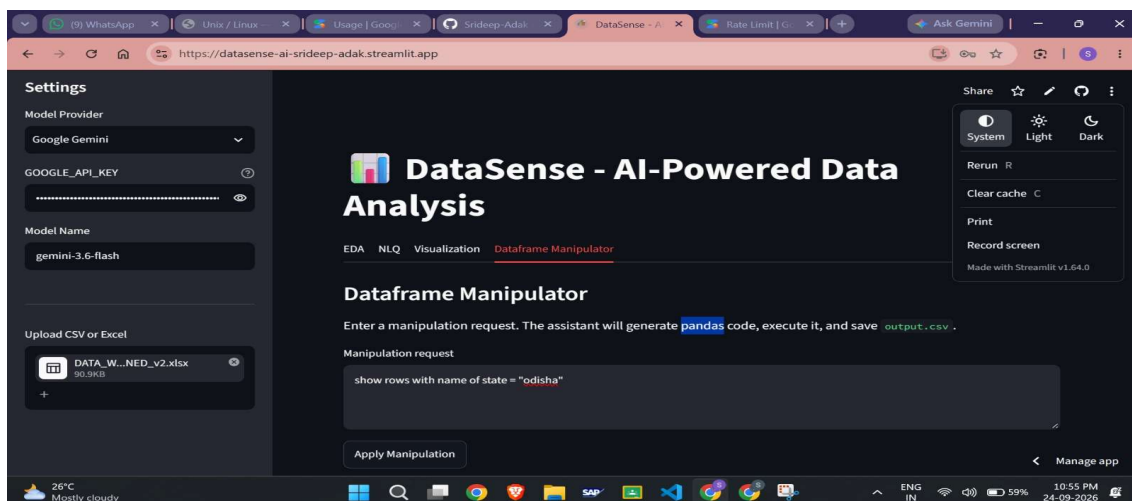


Figure 10.4: Dashboard of dataframe manipulator.

The screenshot shows the 'Dataframe Manipulator' interface. On the left, the 'Settings' panel is visible with 'Model Provider' set to 'Google Gemini', 'GOOGLE_API_KEY' (redacted), 'Model Name' set to 'gemini-3.6-flash', and 'Upload CSV or Excel' showing a file named 'DATA_W...NED_v2.xlsx' (90.9KB). The main area has a title 'Dataframe Manipulator' and a description: 'Enter a manipulation request. The assistant will generate pandas code, execute it, and save output.csv.' Below this is a 'Manipulation request' input box containing the text 'show rows with name of state = "odisha"'. An 'Apply Manipulation' button is below the input box. The result is a table with 6 rows and 6 columns. The first column is an index (0-5), the second is 'Name of State' (all 'ODISHA'), the third is 'Name of District' (Angul, Balasore, Bargarh, Bhadrak, Bolangir, Boudh), the fourth is 'recharge_monsoon', the fifth is 'Recharge from other sources During Monsoon Season', and the sixth is 'Recharge from'. The bottom status bar shows '26°C Mostly cloudy' and system icons.

	Name of State	Name of District	recharge_monsoon	Recharge from other sources During Monsoon Season	Recharge from
0	ODISHA	Angul	40918.06		8016.18
1	ODISHA	Balasore	63946.34		13973.34
2	ODISHA	Bargarh	41040.64		12215.16
3	ODISHA	Bhadrak	17577.53		11087.14
4	ODISHA	Bolangir	46705.54		5498.65
5	ODISHA	Boudh	15719.95		3213.73

Figure 10.5: AI-generated dataframe manipulator showing information of Odisha state only.

The screenshot shows the 'Natural Language Query' dashboard. The 'Settings' panel on the left is identical to the previous figure. The main area has a title 'DataSense - AI-Powered Data Analysis' with a bar chart icon. Below the title are tabs: 'EDA', 'NLQ' (selected), and 'Dataframe Manipulator'. The 'Natural Language Query' section has a description 'Ask a question about your data' and a large input box. To the right of the input box is a 'Suggestions' section with a 'Generate NLQ Suggestions' button. Below the input box is a 'Run NLQ' button. The bottom status bar shows '27°C Cloudy' and system icons.

Figure 10.6: Natural Language query dashboard.

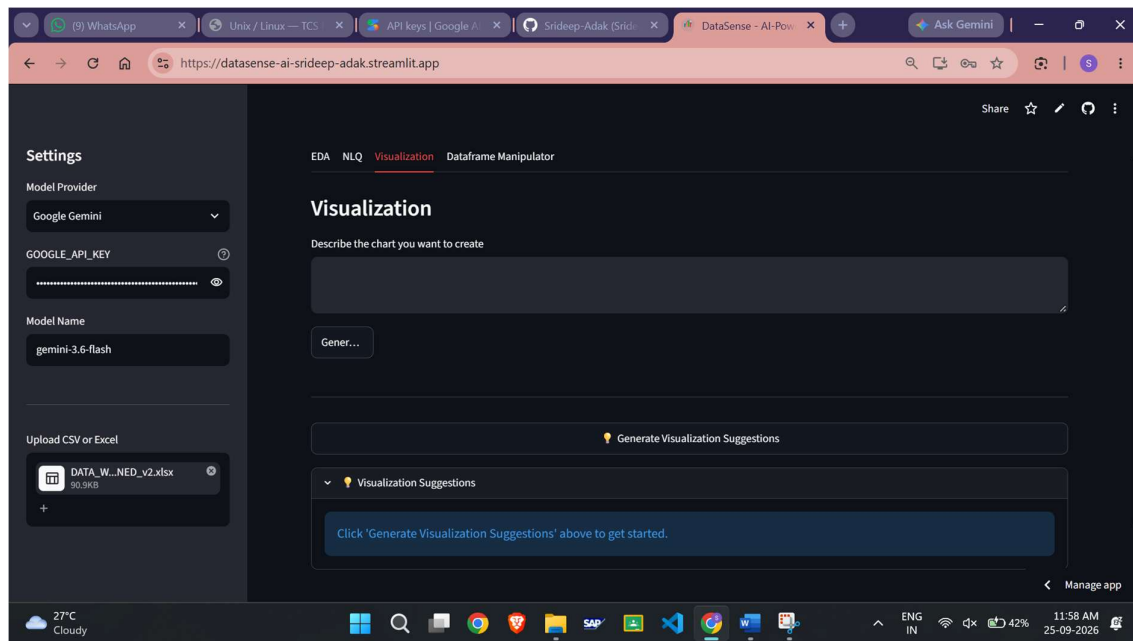


Figure 10.7: Visualization dashboard.

11. Challenges and Learnings

Challenge	Problem encountered	How it was handled
Python and dependency compatibility	The deployed environment used Python 3.14 and the LangChain/Pydantic import chain failed during type-annotation evaluation.	Pinned compatible dependency ranges and moved the deployment runtime toward Python 3.12.
LangChain experimental APIs	The Pandas dataframe agent depends on a compatible combination of LangChain, LangChain Core and experimental packages.	Kept the LangChain packages aligned and treated the agent as a separate integration layer.
Dynamic code execution	AI-generated dataframe operations can execute Python code and therefore require careful handling.	Added execution guardrails, code transparency and deployment caution.
Ambiguous natural-language requests	Users may ask questions without naming the required column or filter.	Added data-aware suggestions and clear feedback so users can refine requests.
Large datasets	In-memory Pandas processing can become slower and consume more memory as file size increases.	Documented file-size expectations and identified scalable processing as future work.

Key learning: an AI data application is not only an LLM integration. Reliable data loading, deterministic dataframe operations, execution safety, dependency management, testing and transparent outputs are equally important.

12. Future Improvements

- Add database connectivity for PostgreSQL, MySQL and cloud data warehouses.
- Introduce persistent conversation history for follow-up questions over the same dataset.
- Add stronger sandboxing for generated Python execution.
- Support larger datasets through chunked or distributed processing.
- Add automated PDF/Excel report generation.
- Add user authentication and access controls for shared deployments.
- Improve automated insight generation and dashboard-style summaries.

13. Conclusion

DataSense-AI demonstrates how natural-language interfaces and large language models can make tabular data analysis more accessible. The application combines Streamlit, Pandas, LangChain and multiple LLM providers to provide automated EDA, natural-language querying, visualization generation and dataframe manipulation in a single workflow.

The project also highlights practical engineering concerns that accompany AI applications: safe execution of generated code, dependency compatibility, transparent generated operations, file-size limitations and objective testing. With measured test results and screenshots added, the system can serve as a practical portfolio project demonstrating both data-analysis and Generative AI application development skills.

14. References and Links

Item	Details
GitHub repository	https://github.com/Srideep-Adak/DataSense-AI
Documentation used	LangGraph docs, ChromaDB docs, RAGAS docs, Open-Meteo API docs, Frankfurter API docs, Streamlit docs
Live Project Link	https://datasense-ai-srideep-adak.streamlit.app